3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

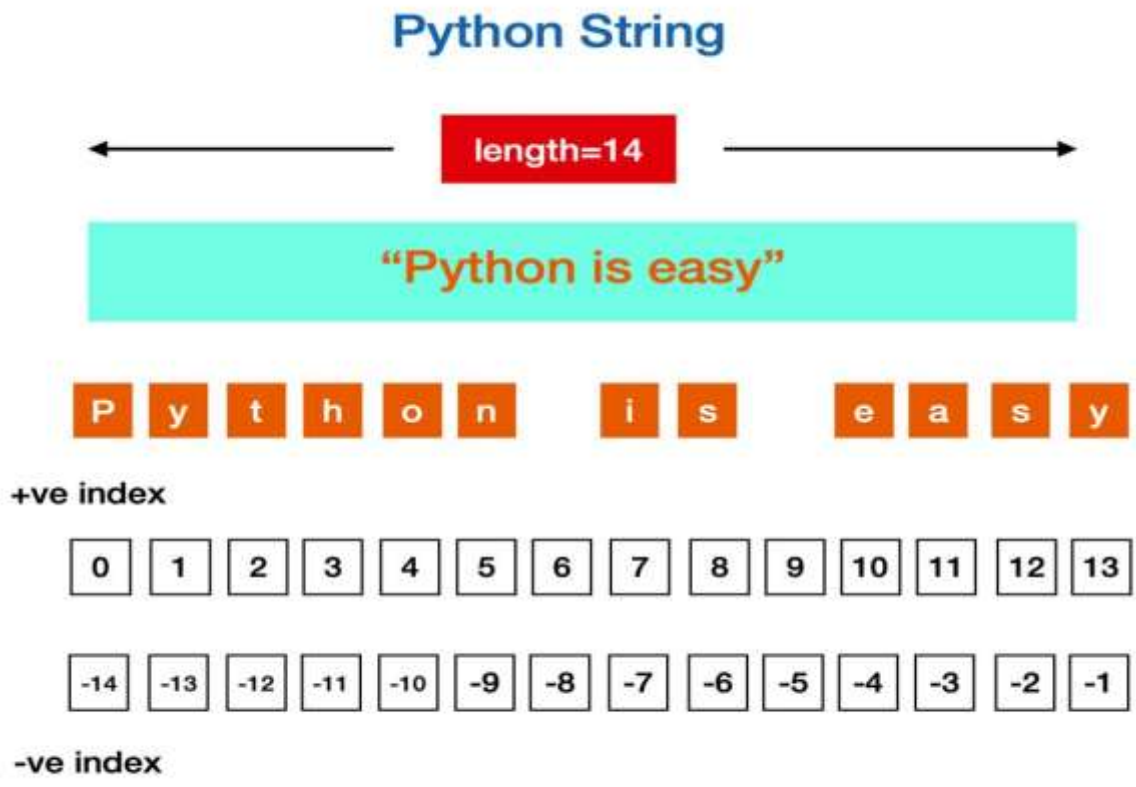
Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 4 – Strings & Advanced String Operations

FOUNDATION



Learning Objectives

By the end of this class, students will be able to:

- Understand what strings are in Python.
- Create and access strings.
- Understand indexing and slicing.
- Use common and advanced string methods.
- Format strings using modern Python techniques.
- Search, replace and split text.
- Remove unwanted spaces and characters.
- Count words and characters.
- Build a practical **Text Processing Program**.

1. What is a String?

A **string** is a sequence of characters enclosed inside quotation marks.

Examples:

name = "Aisha"

city = 'Hyderabad'

course = "Advanced Python"

A string can contain:

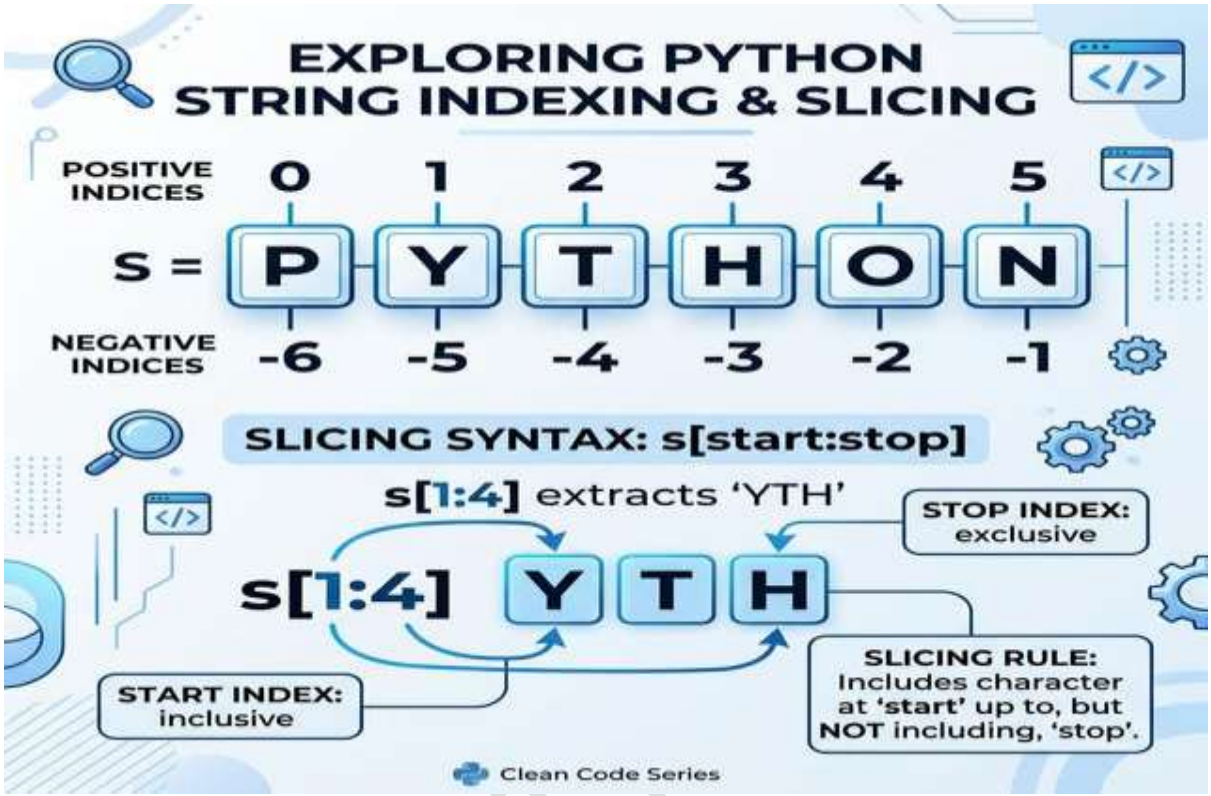
- Letters
- Numbers
- Spaces
- Symbols
- Special characters

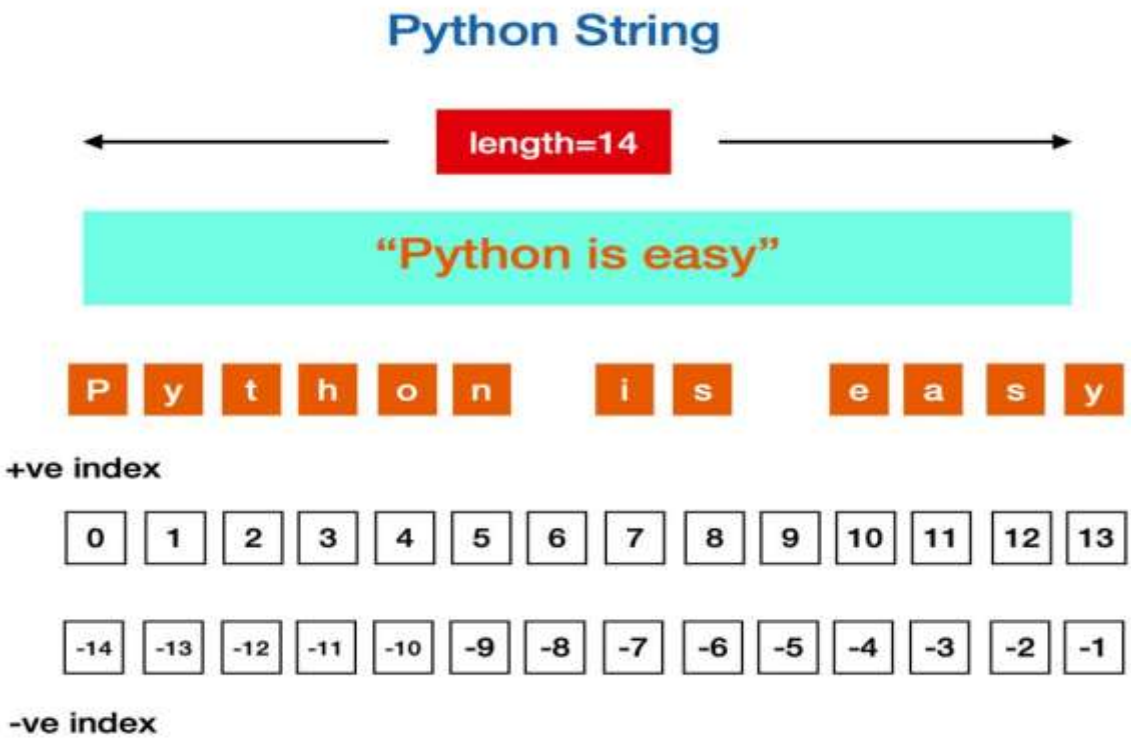
Example:

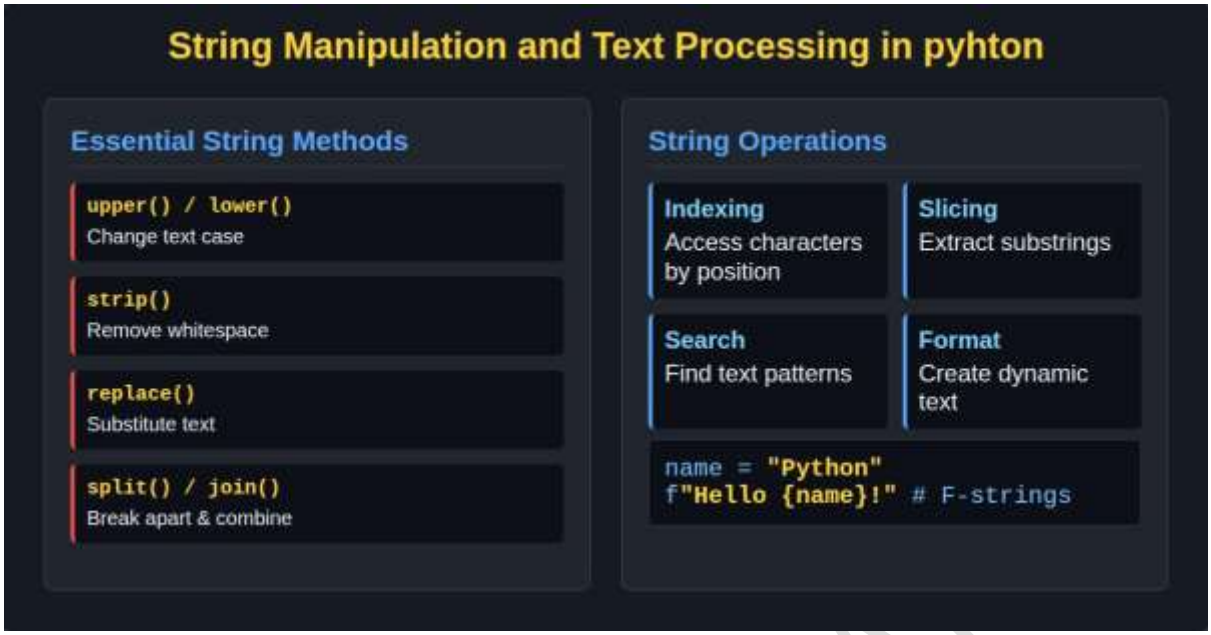
3DWEBSOFT FOUDATION

student_id = "STU1025"

Even though 1025 contains numbers, "STU1025" is a **string**.







2. Creating Strings

Python allows single, double and triple quotation marks.

Single Quotes

name = 'Rahul'

Double Quotes

name = "Rahul"

Triple Quotes

Triple quotes are useful for multi-line text.

message = """Welcome to

Advanced Python

Programming"""

Output:

Welcome to

Advanced Python

Programming

3DWEBSOFT FOUDA

3. Strings and Characters

A string is made up of individual characters.

Consider:

word = "PYTHON"

We can visualize it as:

P Y T H O N

↓ ↓ ↓ ↓ ↓ ↓

0 1 2 3 4 5

Python starts counting positions from **0**.

This position is called an **index**.

4. String Indexing

We can access individual characters using an index.

word = "PYTHON"

print(word[0])

print(word[1])

print(word[5])

Output:

P

Y

N

Important

P Y T H O N

0 1 2 3 4 5

3DWEBSOFT FOUDATION

The first character is always at index 0.

5. Negative Indexing

Python also allows negative indexes.

P Y T H O N

-6 -5 -4 -3 -2 -1

Example:

```
word = "PYTHON"


print(word[-1])
print(word[-2])
```

Output:

N

O

Negative indexing is useful when we want to access characters from the end.

6. String Slicing

Slicing allows us to extract part of a string.

Syntax:

```
string[start:end]
```

Example:

```
word = "PYTHON"


print(word[0:3])
```

Output:

3DWEBSOFT FOUDATION

PYT

The end index is **not included**.

More examples

word = "PYTHON"

print(word[1:4])

print(word[:3])

print(word[3:])

Output:

YTH

PYT

HON

7. Advanced Slicing

Python supports a third value called step.

Syntax:

string[start:end:step]

Example:

word = "PYTHON"

print(word[::2])

Output:

PTO

Every second character is selected.

Reverse a String

3DWEBSOFT FOUDATION

```
word = "PYTHON"


print(word[::-1])
```

Output:

NOHTYP

8. Finding String Length

Use:

len()

Example:

```
message = "Hello Python"


print(len(message))
```

Output:

12

Spaces are also counted as characters.

9. Changing Letter Case

Python provides several useful methods.

.upper()

Converts text to uppercase.

```
name = "python"


print(name.upper())
```

Output:

3DWEBSOFT FOUDATION

PYTHON

.lower()

name = "PYTHON"

print(name.lower())

Output:

python

.title()

name = "advanced python programming"

print(name.title())

Output:

Advanced Python Programming

.capitalize()

text = "python programming"

print(text.capitalize())

Output:

Python programming

10. Searching Inside Strings

.find()

Finds the position of a substring.

text = "I am learning Python"

3DWEBSOFT FOUDATION

```
position = text.find("Python")


print(position)
```

.count()

Counts how many times something appears.

```
text = "Python is easy. Python is powerful."


print(text.count("Python"))
```

Output:

2

11. Replacing Text

The .replace() method replaces one piece of text with another.

```
message = "I love Java"


new_message = message.replace("Java", "Python")


print(new_message)
```

Output:

I love Python

Practical Use

Replacing incorrect or outdated information in text is a common text-processing operation.

12. Splitting a String

3DWEBSOFT FOUDATION

.split() divides a string into a list.

```
sentence = "Python is easy to learn"

words = sentence.split()

print(words)
```

Output:

```
['Python', 'is', 'easy', 'to', 'learn']
```

This is extremely useful for **text processing**.

Split Using a Specific Character

```
data = "apple,banana,mango"

fruits = data.split(",")

print(fruits)
```

Output:

```
['apple', 'banana', 'mango']
```

13. Joining Strings

.join() combines multiple strings.

```
words = ["Python", "is", "powerful"]

sentence = " ".join(words)

print(sentence)
```

3DWEBSOFT FOUDATION

Output:

Python is powerful

Another example:

```
names = ["Asha", "Rahul", "Priya"]


result = ", ".join(names)


print(result)
```

Output:

Asha, Rahul, Priya

14. Removing Unwanted Spaces

The .strip() method removes spaces from the beginning and end.

```
name = "   Aisha   "


clean_name = name.strip()


print(clean_name)
```

Output:

Aisha

.lstrip()

Removes spaces from the left.

.rstrip()

Removes spaces from the right.

These methods are particularly useful when processing user input.

3DWEBSOFT FOUDATION

15. Checking String Content

Python provides methods to check what a string contains.

.isdigit()

value = "12345"

print(value.isdigit())

Output:

True

.isalpha()

name = "Python"

print(name.isalpha())

Output:

True

.isalnum()

Checks whether the string contains only letters and numbers.

student_id = "STU1025"

print(student_id.isalnum())

Output:

True

16. Important String Methods

3DWEBSOFT FOUDATION

Method	Purpose
upper()	Converts to uppercase
lower()	Converts to lowercase
title()	Capitalizes each word
capitalize()	Capitalizes first character
strip()	Removes outer whitespace
replace()	Replaces text
find()	Finds text position
count()	Counts occurrences
split()	Splits into a list
join()	Combines strings
startswith()	Checks beginning
endswith()	Checks ending
isdigit()	Checks digits
isalpha()	Checks letters
isalnum()	Checks letters/numbers
len()	Finds length

3DWEBSOFT FOUDATION

17. String Formatting

String formatting allows us to insert variables into text.

f-Strings

Modern Python commonly uses **f-strings**.

```
name = "Aisha"
```

```
age = 17


print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Aisha and I am 17 years old.

Another Example

```
product = "Laptop"
price = 45000


print(f"The price of the {product} is ₹{price}.")
```

18. Escape Characters

Escape characters allow us to include special characters.

New Line

```
print("Hello\nPython")
```

Output:

Hello

Python

Tab

```
print("Name:\tAisha")
```

Output:

Name: Aisha

Quotation Mark

```
print("She said, \"Hello\"")
```

3DWEBSOFT FOUDATION

19. Strings are Immutable

One important Python concept is that **strings are immutable**.

This means we cannot directly change an individual character.

This causes an error:

```
word = "Python"


word[0] = "J"
```

Instead, create a new string:

```
word = "Python"


new_word = "J" + word[1:]


print(new_word)
```

Output:

Jython

This concept becomes important when students work with advanced data processing.

3DWEBSOFT FOUDATION

Practical Session

20. Practical Activity 1 – Personal Information Processor

```
name = input("Enter your full name: ")

city = input("Enter your city: ")
```

```
name = name.strip().title()

city = city.strip().title()


print(f"Name: {name}")

print(f"City: {city}")

print(f"Name length: {len(name)}")
```

Example

Input:

Enter your full name: aisha khan

Enter your city: hyderabad

Output:

Name: Aisha Khan

City: Hyderabad

Name length: 11

21. Practical Activity 2 – Word Counter

Ask students to create a simple word counter.

```
sentence = input("Enter a sentence: ")


words = sentence.split()


print("Number of words:", len(words))
```

Example

Enter a sentence: Python is easy to learn

3DWEBSOFT FOUDATION

Number of words: 5

22. Practical Activity 3 – Character Counter

```
text = input("Enter some text: ")


print("Total characters:", len(text))
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
```

Challenge

Ask students to count a particular character.

```
text = input("Enter text: ")
character = input("Enter character to count: ")


print("Occurrences:", text.count(character))
```

23. Practical Activity 4 – Email Checker

Students can build a simple **educational** email-format checker.

```
email = input("Enter your email: ").strip()


if "@" in email and "." in email:
    print("Basic email format detected")
else:
    print("Please check the email format")
```

Important

3DWEBSOFT FOUDATION

This is only a basic format check. A production application would require much stronger validation.

24. Main Practical – Text Processing Program

Now students will combine today's concepts into one project.

Project Goal

Create a program that accepts a paragraph and analyzes it.

Program

```
text = input("Enter a paragraph: ")


clean_text = text.strip()


words = clean_text.split()


print("\n----- TEXT ANALYSIS -----")


print("Original text:", text)
print("Clean text:", clean_text)
print("Characters:", len(clean_text))
print("Words:", len(words))
print("Uppercase:", clean_text.upper())
print("Lowercase:", clean_text.lower())
```

25. Add Advanced Text Analysis

Now improve the program.

3DWEBSOFT FOUDATION

```
text = input("Enter a paragraph: ").strip()

words = text.split()

print("\n----- TEXT ANALYSIS -----")

print("Characters:", len(text))
print("Words:", len(words))
print("Sentences:", text.count("."))
print("Python occurrences:", text.lower().count("python"))
print("First word:", words[0] if words else "No words")
print("Last word:", words[-1] if words else "No words")
```

Example

Input:

Python is easy. Python is powerful. Python is useful.

Possible output:

----- TEXT ANALYSIS -----

Characters: 50

Words: 9

Sentences: 3

Python occurrences: 3

First word: Python

Last word: useful.

3DWEBSOFT FOUDATION

26. Interactive Challenge – Text Detective

Give students this sentence:

"Python is powerful and Python is easy to learn."

Ask them:

Question 1

How many times does "Python" occur?

Question 2

How many words are there?

Question 3

Convert the sentence to uppercase.

Question 4

Replace "powerful" with "popular".

Question 5

Reverse the complete string.

Question 6

Find the position of "easy".

Students should write Python code to answer each question.

3DWEBSOFT FOUDATION

27. Advanced Challenge – Word Frequency

Students can create a simple word-frequency analyzer.

```
text = input("Enter a sentence: ").lower()


words = text.split()


word = input("Enter a word to search: ").lower()
```

```
count = words.count(word)


print(f"The word '{word}' appears {count} time(s).")
```

Example

Enter a sentence: Python is easy and Python is powerful

Enter a word to search: python

The word 'python' appears 2 time(s).

This introduces students to the idea of **text analysis**, which is useful later in AI and Natural Language Processing.

28. Real-World Applications

String processing is used in:

AI & NLP

Processing human language.

Search Engines

Searching keywords in documents.

Chatbots

Analyzing user messages.

Email Systems

Processing email addresses and messages.

Data Science

Cleaning and preparing text data.

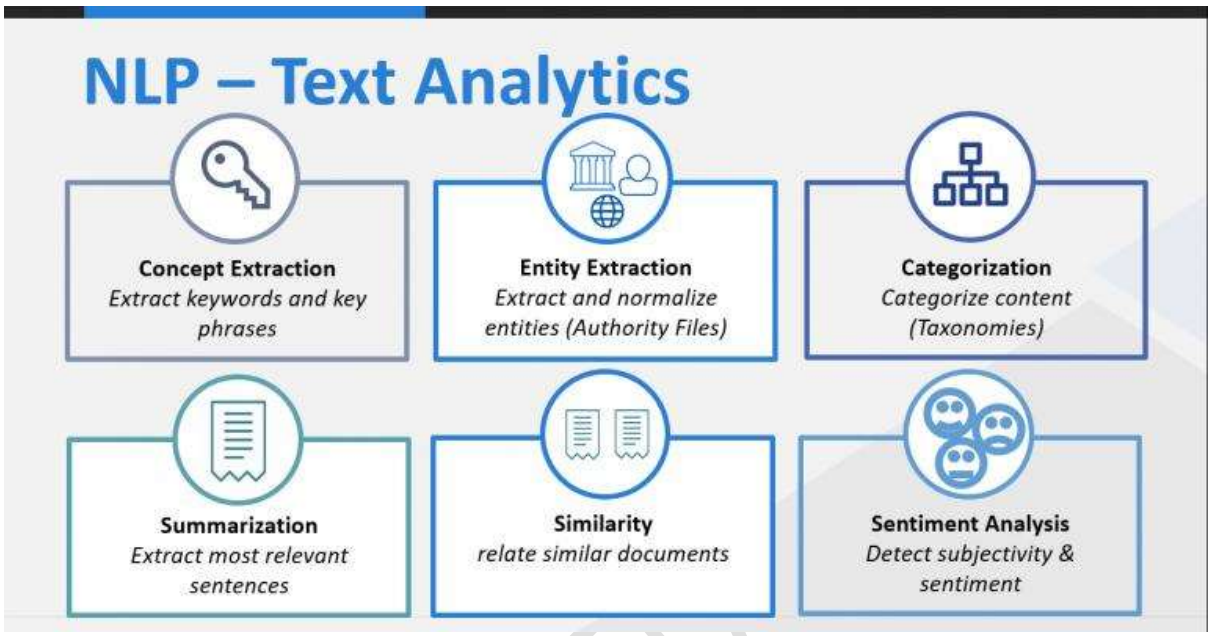
News Applications

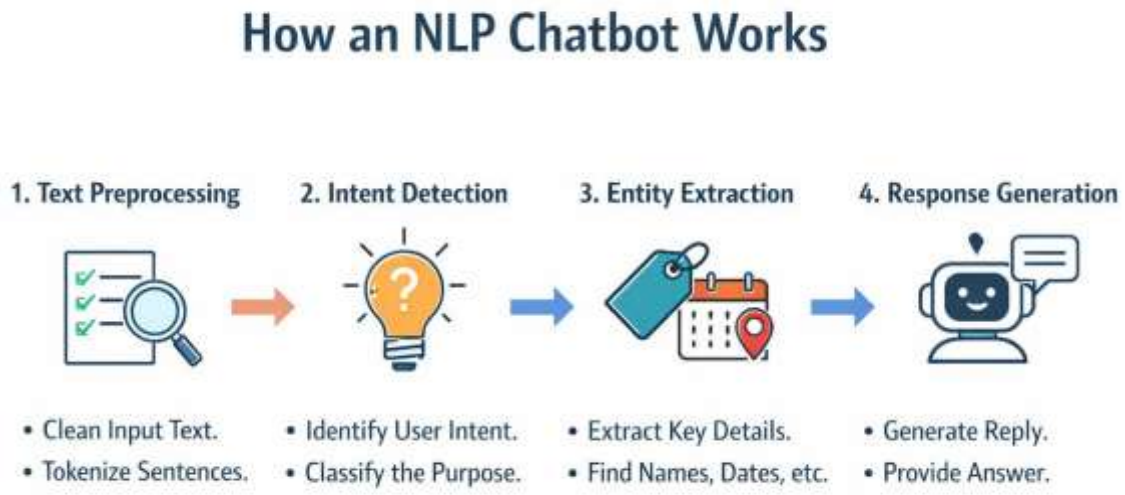
3DWEBSOFTFOUDATION

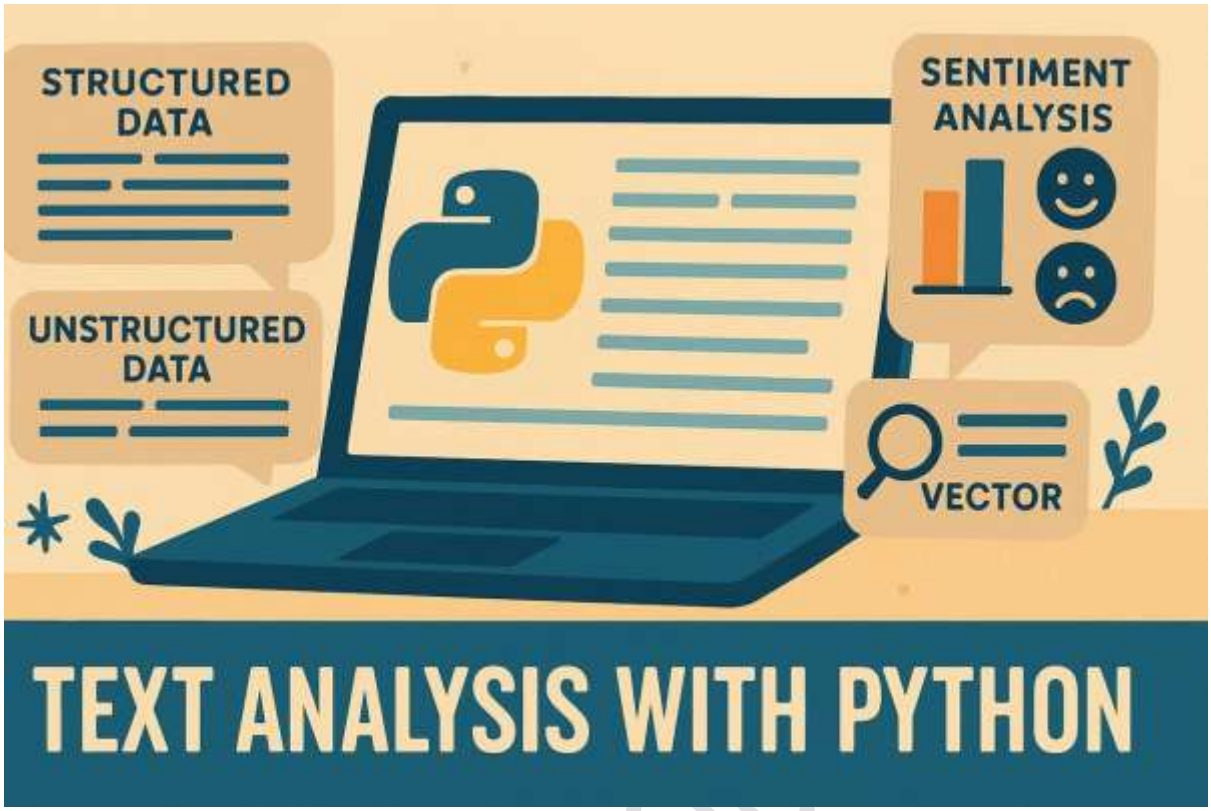
Analyzing headlines and articles.

Cybersecurity

Analyzing text and logs for defensive security purposes.







Quick Quiz

Q1. What is a string?

Answer: A sequence of characters.

Q2. What is the first index of a Python string?

Answer: 0

Q3. Which method converts text to uppercase?

Answer: upper()

Q4. Which method removes whitespace from both ends?

Answer: strip()

Q5. Which method splits a string into a list?

Answer: split()

Q6. Which method combines a list of strings?

Answer: join()

Q7. What does len() do?

Answer: Returns the number of characters/items.

Q8. What does [::-1] commonly do to a string?

Answer: Reverses the string.

Q9. Are Python strings mutable?

Answer: No. Strings are immutable.

30. Homework

Basic

Write programs to:

1. Convert a name to uppercase.
2. Convert a sentence to lowercase.
3. Count the number of characters.
4. Count the number of words.
5. Replace one word with another.
6. Reverse a string.

Intermediate

Create a **Student Name Formatter**:

Input:

Enter first name:

Enter middle name:

Enter last name:

Output:

Full Name:

3DWEBSOFT FOUDATION

Initials:

Number of Characters:

Uppercase Name:

Lowercase Name:

Advanced

Create a **Text Analyzer** that accepts a paragraph and displays:

- Number of characters
- Number of words
- Number of sentences
- Number of vowels
- Number of digits
- Number of spaces
- Most frequently searched word
- Uppercase version
- Lowercase version

2-Hour Class Plan

Time	Activity
0–10 min	Revision of variables & data types
10–25 min	Introduction to strings
25–40 min	Indexing & negative indexing
40–55 min	Slicing
55–75 min	String methods
75–90 min	String formatting

3DWEBSOFT FOUDATION

Time	Activity
90–105 min	Practical text processing
105–115 min	Text Detective challenge
115–120 min	Quiz & homework

Student Quick Reference

len(text) → Length

text.upper() → Uppercase

text.lower() → Lowercase

text.title() → Title Case

text.strip() → Remove outer spaces

text.find("x") → Find position

text.count("x") → Count occurrences

text.replace(a, b) → Replace text

text.split() → String → List

" ".join(words) → List → String

text.startswith(x) → Check beginning

text.endswith(x) → Check ending

text[::-1] → Reverse

3DWEBSOFT FOUDATION

Key Takeaway

Strings are one of the most important data types in Python. Mastering string operations gives students a strong foundation for data processing, automation, AI, Natural Language Processing and real-world application development.